

Mojo

Введение, основы языка программирования

5030102/30202

Григорьева А.

Шевцов Д.

Шильдт Д.

О Mojo



Первый релиз

август 2026 года



Назначение

Язык системного
программирования



Универсальность

Применяется на низком и на
высоком уровнях

```
1  def main() raises:
2      var name: String = "Mojo"
3      var greeting: String = "Hi, " + name + "!"
4      print(greeting)
```

```
Hi, Mojo!
```

Общие сведения о языке

1. Изначально задумывался как супермножество для Python.
2. Python-подобный синтаксис;
3. Использует (и расширяет) синтаксис Python и интегрируется с существующим кодом Python.
4. Компилируемый: используется JIT-компиляция и АОТ-компиляция.
5. Гибридная типизация.

Запуск языка

Поддержка операционных систем, окружение и версии



Нативная поддержка

Полная нативная поддержка для систем **Linux** и **macOS**.



Решение для Windows

Для работы в Windows используется среда подсистемы **WSL2**.



Пакетные менеджеры

Часто используемые менеджеры: **pixi**, **pip**, **uv**, **conda**.



Версии языка

Существуют **Stable** и **Nightly** версии языка.

Запуск языка:

(на примере менеджера uv)

1. Установка uv:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

2. Создание проекта:

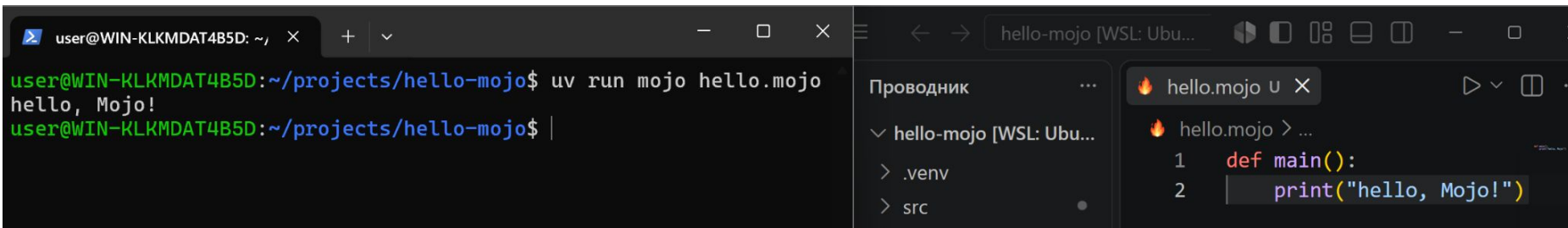
```
uv init hello-mojo
```

```
cd hello-mojo
```

3. Добавьте тожо в зависимости:

```
uv add Mojo
```

4. Запустите программу:



The screenshot shows a terminal window with the following content:

```
user@WIN-KLKMDAT4B5D: ~/projects/hello-mojo$ uv run Mojo hello-mojo
hello, Mojo!
user@WIN-KLKMDAT4B5D: ~/projects/hello-mojo$
```

On the right side of the terminal, there is a file explorer pane showing the directory structure:

```
Проводник
└─ hello-mojo [WSL: Ubu...
  └─ .venv
  └─ src
```

Below the file explorer, there is a code editor pane showing the contents of the `hello.mojo` file:

```
hello.mojo > ...
1 def main():
2     print("hello, Mojo!")
```

Запуск языка: Docker

1. Создать Dockerfile, добавить базовый конфиг:

```
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y curl ca-certificates && rm -rf /var/lib/apt/lists/*

RUN curl -fsSL https://pixi.sh/install.sh | sh
ENV PATH="/root/.pixi/bin:${PATH}"

WORKDIR /app
RUN pixi init . -c https://conda.modular.com/max -c conda-forge \
    && pixi add mojo

COPY . .
CMD ["pixi", "run", "mojo", "run", "main.mojo"]
```

2. Сборка образа: `docker build -t mojo-app .`
3. Запуск: `docker run --rm mojo-app`

Ввод/вывод

1.

```
user@WIN-KLKMDAT4B5D:~/projects/hello-mojo$ uv run mojo hello.mojo
Who are you? student
Hi, student!
user@WIN-KLKMDAT4B5D:~/projects/hello-mojo$ |
```

```
hello.mojo u X
hello.mojo > ...
1 def main() raises:
2     var name: String = input("Who are you? ")
3     var greeting: String = "Hi, " + name + "!"
4     print(greeting)
```

Встроенная функция `input()` принимает необязательный аргумент типа `String` и возвращает строку

2.

```
user@WIN-KLKMDAT4B5D:~/projects/hello-mojo$ uv run mojo hello.mojo
"text from console"
hello.mojo
text from console
user@WIN-KLKMDAT4B5D:~/projects/hello-mojo$ |
```

```
hello.mojo u X
hello.mojo > ...
1 from std.sys import argv
2
3 def main() raises:
4     var args = argv()
5     for arg in args:
6         print(arg)
```

`argv()` получает список аргументов командной строки, переданных CLI.

Ввод/вывод: файлы

```
1  from std.os import path
2
3  def main() raises:
4      var filename: String = "example.txt"
5
6      with open(filename, "w") as f:
7          f.write("Привет из Моjo!\n")
8          f.write("Вторая строка.\n")
9
10     if path.exists(filename):
11         print("Файл создан:", filename)
12
13     with open(filename, "r") as f:
14         var content = f.read()
15         print("Содержимое файла:")
16         print(content)
```

Problems Output Debug Console Terminal Ports

MojoTest\$ uv run mojo main.mojo

Файл создан: example.txt
Содержимое файла:
Привет из Моjo!
Вторая строка.

Типы данных



Все значения в Mojo имеют тип данных.

Переменные

```
def main() raises:
  var a: Int = 10
  var b: Int
  b = 3
  #var b ERROR
  c = 10 #WARNING
  print("+:", a + b)
  print("//:", a // b)
  print("%:", a % b)
  print(">:", a > b)
  print("&:", (a > b) and (b > 0))
  var i: Int = 42
  var f: Float64 = Float64(i)      # Int -> Float64
  var s: String = String(i)       # Int -> String
```

```
+: 13
//: 3
%: 1
>: True
&: True
```

```
var s1: String = "hello"
var s2 = s1
s2 += " World"
print("s1:", s1)
print("s2:", s2)

consume(s1^)      # ^ передаёт владение
# print(s1)       # ERROR
print("s2:", s2)
```

```
s1: hello
s2: hello World
Consumed: hello
s2: hello World
```

Переменные

- Блочная область видимости.
- Значение переменной удаляется сразу после последнего использования, само имя - при выходе из области видимости.

```
1  def main():
2      var a = 10
3
4      if a == 10:
5          var b = 20
6          print(a)    # внешняя переменная доступна
7          print(b)    # переменная внутри своего блока доступна
8
9      # print(b)      # Ошибка: использование неопределенной переменной "b"
10     print(a)        # доступна
```

Функции

Основные особенности:

- Все функции начинаются с ключевого слова `def`
- У простейшей функции есть имя, пустые скобки и тело
- Функции должны содержать возвращаемый тип возвращаемого значения

ПРИМЕР ПРОСТЕЙШЕЙ ФУНКЦИИ

```
def do_nothing():  
    pass
```

Сигнатура функций:

```
def name[parameter-list](argument-list) raises  
    -> ReturnType where constraint:  
    body
```

```
1  # [size: Int] — параметр  
2  # (x: Int)    — аргумент  
3  
4  ✓ def add(a: Int, b: Int) -> Int: # только аргументы  
5      |   return a + b  
6  
7  ✓ def get_max_size[size: Int]() -> Int: # только параметр  
8      |   return size * 2  
9  
10 ✓ def make_array[size: Int](x: Int) -> Int: # и аргумент,  
11      |   return x * size                    # и параметр
```

Функции: аргументы и параметры

	Параметры [T: type]	Аргументы (arg: Type)
Когда известны	Во время компиляции	Во время выполнения
Синтаксис	[]	()
Что делает компилятор	Генерирует специализированные версии функции	Передает значения в готовую функцию

Функции: входные данные. Соглашение о передаче аргументов

```
def double_it(mut x: Int):  
  x *= 2
```

```
def consume(var s: String):  
  s += "!"; print(s)  
def main():  
  var greeting = "Hello"  
  consume(greeting) # Hello!  
  print(greeting) # Hello
```

```
def make_int(out result: Int):  
  result = 42  
def main():  
  var x = make_int()  
  print(x) # 42
```

```
struct Resource:  
  var handle: Int  
  def __deinit__(deinit self):  
    _release(self.handle)
```

```
def get_first[T: Copyable](ref data: List[T])  
-> ref[data[0]] T:  
  return data[0]  
def main():  
  ref first = get_first(data) # mutable
```

```
def length[T: Copyable](s: List[T]) ->  
Int:  
  return len(s)
```

Функции: рекурсия

Принцип работы такой же, как в Python

```
1  def factorial(n: Int) -> Int:
2      if n <= 1:
3          return 1
4      return n * factorial(n - 1)
5
6  def main():
7      print(factorial(5)) # 120
```

Функции: замыкание.

```
def name[parameter-list](argument-list) raises {capture-list} -> ReturnType:  
  body
```

```
1  def main():  
2      var multiplier = 3  
3  
4      def scale(x: Int) {imm multiplier} -> Int:  
5          return x * multiplier  
6  
7      print(scale(5))  # 15
```

Конструкции потока управления

1. If:

```
def main() raises:
    score = 65
    if score >= 90:
        print("A")
    elif score >= 75:
        print("B")
    elif score >= 60:
        print("C")
    else:
        print("F")
#Output C
var n: Int = 7
var parity: String = "even" if n % 2 == 0 else "odd"
print(n, "-", parity) #7 - odd
```

```
var age: Int = 20
var has_ticket: Bool = True
if age >= 18 and has_ticket:
    print("Access")
elif age >= 18 and not has_ticket:
    print("Need ticket")
else:
    print("Need 18 years")
#Output Access
```

Конструкции потока управления

2. While:

```
def main() raises:
    var n: Int = 0
    while n < 3:
        print("n =", n)
        n += 1
    else:
        print("W/o break\n")

    var k: Int = 0
    while k < 5:
        if k == 2:
            print("k = 2!!")
            break
        print("k =", k)
        k += 1
    else:
        print("W break")
```

```
n = 0
n = 1
n = 2
W/o break

k = 0
k = 1
k = 2!!
```

```
var p: Int = 0
while p < 6:
    p += 1
    if p % 2 == 0:
        continue
    print("odd -", p)
else:
    print("W continue\n")

var m: Int = 10
while m < 5:
    print("m =", m)
    m += 1
else:
    print("W False")
```

```
odd - 1
odd - 3
odd - 5
W continue

W False
```

Конструкции потока управления

3. For

- Выполняет последовательно итерации по типам, у которых реализованы методы `__iter__()`, `__next__()` и `__len__()`

```
for i in range(5):
    print("i =", i)
var x = ["A", "B", "C"]
for i in x:
    print(i)
var y = ["a", "b", "c", "d"]
for i in y:
    if i == "d":
        print("Find d")
        break
for i in range(6):
    if i % 2 == 0:
        continue
    print("odd -", i)
```

```
i = 0
i = 1
i = 2
i = 3
i = 4
A
B
C
Find d
odd - 1
odd - 3
odd - 5
```

```
for i in range(5):
    print("i =", i)
else:
    print("w/o break")
var values = [1, 4, 7, 3, 6, 11]
for ref value in values:
    if value % 2 != 0:
        value -= 1
for i in values:
    print(i)
```

```
i = 0
i = 1
i = 2
i = 3
i = 4
w/o break
0
4
6
2
6
10
```